26900160 Reg.No:.....

Name:.....

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (Honours)/ MGU-BBA (Honours)/ MGU-BCA (Honours)

REGULAREXAMINATION October 2026

Fifth SEMESTER

Discipline Specific Core Course (DSC) - MG5DSCCFS301 - SECURITYANALYSIS USING PYTHON

(2024 ADMISSION ONWARDS)

Duration: 1 hour 30 minutes Maximum Marks: 50

Remember(K), Understand(U), Apply(A), Analyse(An), Evaluate(E), Create(C), Skill(S), Interest(I)and Appreciation(Ap)

Students should attempt at least one question from each course outcome to enhance their overalloutcome attainability.

Part A

Very Short Answer Questions

Answer any **10** questions

Each question carries **1** marks

1. Explain why lists are called mutable objects in Python. [U] / [CO1]

   Lists are mutable because their elements can be modified, added or removed after the list is created.

   Example: a = [10, 20]; a[0] = 5 changes the list to [5, 20].

2. Explain how Python evaluates an expression. [U] / [CO1]

   Python evaluates an expression according to operator precedence and associativity to produce a value. For example, 2 + 3 * 4 evaluates to 14 because multiplication occurs before addition.

3. Distinguish between in and not in operators when used with strings. [U] / [CO1]

   The in operator checks whether a substring exists in a string it returns True if found, where as not in checks whether it is absent from a string. It returns True if not found.

   Example - text = "Python Programming"
   print("Python" in text)
   print("Java" not in text)

4. Differentiate between for and while loops. [U] / [CO2]

   A for loop iterates over the items of a sequence or other iterable. A while loop repeats as long as its condition remains true.

5. Define constructor. [K] / [CO2]

   A constructor is used to initialize an object when it is created. In Python, the __init__() method commonly performs this initialization.

6. Define recursion. [K] / [CO2]

   Recursion is a programming technique in which a function calls itself to solve a problem. A base case is needed to stop the recursive calls.

7. Differentiate between local and global variables. [U] / [CO2]

   A local variable is defined inside a function and has local scope. A global variable is defined at module level and can be accessed throughout the modules.

8. List the basic components of a function definition. [K] / [CO2]
   The basic components are the def keyword, function name, parentheses, optional parameters, colon and function body. A return statement may also be included.

9. Describe the difference between an expression and a statement in Python. [U] / [CO1]
   An expression produces a value, such as 5 + 3. A statement performs an action, such as x = 8 or print(x).
10. Express the role of SMTP in email communication. [U] / [CO3]
    SMTP stands for Simple Mail Transfer Protocol. It is used to send and relay email messages between email clients and mail servers or between mail servers.

11. Describe the steps to develop an efficient IP scanner in Python. [An] / [CO3]
    Define the authorized IP range, select a scanning method, use Python networking libraries, check host or port responses, handle timeouts and exceptions, and display results efficiently.
12. Explain what is ping sweep. [An] / [CO3]
    A ping sweep is a network discovery technique used to identify potentially active hosts within an IP address range by sending probes and examining the responses.

[1x10 = 10]

PART B

Short Answer Questions

Answer any **4** questions

Each question carries **5** marks

13. Explain how to create and access elements of a dictionary with suitable examples.[U] /[CO1]
    A dictionary is a mutable Python data structure that stores data as key-value pairs. Each key is used to access its corresponding value. Dictionaries are useful for storing related information, such as student details, employee records and configuration settings.A dictionary can be created using curly braces or the dict() function.

```
student = {

  "name": "Anu",

   "age": 20,

"course": "BSc"

    }

  print(student["name"])

 print(student["age"])

    print(student.get("course"))
```

14. Illustrate the procedure for creating and executing a simple Python program. [U] / [CO1]

    Python is a high-level programming language used to develop applications, automate tasks and perform data processing. The procedure for creating and executing a program begins with installing Python and opening an editor such as IDLE. The programmer writes the instructions, saves the file with

a .py extension and executes it using the editor or command line.

```
a = 10
b = 20
total = a + b
print("Sum =", total)
```

15. Illustrate the flow of execution of a simple Python program. [U] / [CO2]

The flow of execution refers to the order in which statements are executed in a program. Python normally executes statements from top to bottom. Conditional statements, loops and function calls can change this order.

```
a = 10

b = 5

if a > b:

  print("a is greater")

else:

    print("b is greater")

    print("Program completed")
```

16. Explain relational operators with an example. [U] / [CO2]

Relational operators compare two values and return a Boolean result, either True or False. They are commonly used in conditional statements and loops.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	8 > 4	True
<	Less than	2 < 7	True
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	3 <= 2	False

Example program:

```
a = 10
b = 20
print(a == b)
print(a < b)
print(a != b)
```

17. Differentiate between a module and a package. [U] / [CO2]

A module is a Python file containing reusable code, such as functions, classes and variables. A

package is a way to organize related modules into a directory hierarchy. A module helps separate a program into reusable units, while a package organizes multiple related modules into a larger structure.

Module	Package
Usually a single Python `.py` file	Usually a directory containing modules and possibly subpackages
Contains functions, classes and variables	Groups related modules
Imported using statements such as `import calculator`	Modules can be imported using dotted names
Example: `calculator.py`	Example: `my_package/`
Suitable for organizing a unit of code	Suitable for organizing a larger application

Example of a module

Create `calculator.py`:

```
def add(a, b):
    return a + b
```

Use it in another file:

```
import calculator

print(calculator.add(5, 3))
```

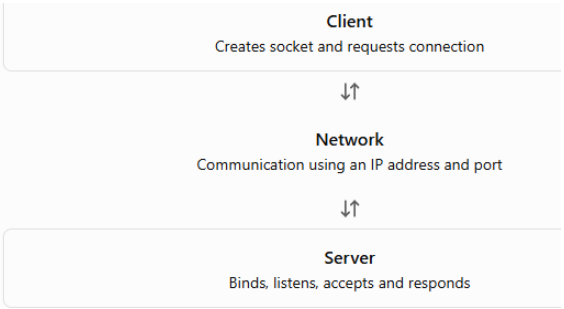
A package may contain files organized as follows:

```
my_package/
__init__.py
calculator.py
student.py
```

18. Outline the client-server model used in socket programming. [U] / [CO3]

The client-server model is a communication model in which a client requests a service and a server provides the service. Python socket programming allows applications to communicate over a network. A socket acts as an endpoint for communication.In TCP socket programming, the server creates a socket and binds it to an IP address and port number. It listens for connection requests and accepts a client connection. The client creates its own socket and connects to the server. After establishing the connection, they exchange data and close the connection when communication is complete.

Basic flow:



Example: A web browser acts as a client when it requests a webpage from a web server. The server processes the request and sends the webpage data back to the browser. **[5*4=20]**

Part C – Essay Questions

Answer any 2 questions. Each question carries 10 marks.

19. Discuss sets in Python and explain their creation, properties, and common operations. [U] / [CO1]

A set in Python is an unordered collection of unique elements. It does not allow duplicate values and is useful for performing mathematical set operations.

Creation of a Set:

A set is created using curly braces `{}` or the `set()` function.

```
A = {10, 20, 30, 40}
B = {30, 40, 50, 60}
```

Properties of Sets: Sets contain only unique elements, Sets are unordered collections, Elements can be added or removed,Sets do not support indexing.

Common Set Operations: - Union (`|`): Combines elements from both sets, Intersection (`&`): Returns common elements.Difference (`-`): Returns elements present in the first set but not the second. Symmetric difference (`^`): Returns elements present in either set, but not in both.

```
A = {10, 20, 30, 40}

B = {30, 40, 50, 60}

print("Union:", A | B)

print("Intersection:", A & B)

print("Difference:", A - B)

print("Symmetric Difference:", A ^ B)
```

20. List common built-in exceptions and explain how exceptions are handled. Describe the method of defining a new exception in Python. [U] / [CO2]

An exception is an error that occurs during the execution of a program and interrupts its normal flow. Python provides several built-in exceptions to indicate different types of errors.

Common built-in exceptions are:

1. ZeroDivisionError: Occurs when a number is divided by zero.
2. TypeError: Occurs when an operation is performed on an inappropriate data type.
3. ValueError: Occurs when a function receives an invalid value.
4. NameError: Occurs when an undefined variable is used.
5. IndexError: Occurs when accessing an invalid list index.
6. KeyError: Occurs when a dictionary key does not exist.
7. FileNotFoundError: Occurs when the specified file cannot be found.
8. ImportError: Occurs when an imported module cannot be loaded.

Exception Handling in Python

Exception handling is a method of handling runtime errors so that the program can continue or terminate gracefully instead of stopping unexpectedly.

Python uses the following keywords for exception handling:

- try: Contains the code that may produce an exception.
- except: Handles the exception.
- else: Executes if no exception occurs in the try block.
- finally: Executes whether an exception occurs or not.
- raise: Used to generate an exception manually.

Example program:

```
try:
a = int(input("Enter a number: "))
b = int(input("Enter another number: "))
result = a / b

print("Result:", result)

except ZeroDivisionError:
print("Cannot divide by zero")
except ValueError:
print("Please enter valid numbers")
finally:
print("Program completed")
```

Defining a New Exception in Python

Python allows programmers to define their own exceptions by creating a class derived from the built-in Exception class. These are called user-defined exceptions. The raise keyword is used to generate the custom exception.

Example program:

```
class AgeError(Exception):
pass
age = int(input("Enter your age: "))
try:
if age < 18:
raise AgeError("Age must be 18 or above")
print("Eligible to vote")
except AgeError as e:
print(e)
```

21. Discuss the scope and objectives of penetration testing with examples. [U] / [CO3]

Penetration testing (pentesting) is the process of testing the information security measures of a company. It includes testing the company's network, database, website, public-facing servers, security policies, and other systems specified by the client.At the end of penetration testing, the tester prepares a detailed report containing the weaknesses, vulnerabilities, risk levels, and possible solutions.

Scope of Penetration Testing

The scope of penetration testing defines the systems and components to be tested. The following points should be considered while defining the scope:

1. Client Consultation: The scope should be developed in consultation with the client. For example, if a client wants to test the entire network, the tester must identify the network components to be included.
2. Time, People, and Money: The tester should consider the available time, manpower, and budget.

3. Test Boundaries: The boundaries of the test should be clearly defined based on the agreement between the tester and the client.
4. Changes in Business Practices: Changes such as adding a subnet, installing new system components, or modifying a web server may change the scope of testing.

5. Components to Be Tested: The main components include:

- Routers, switches, and gateways.
- Public-facing systems, websites, DMZ, email servers, and remote systems.
- DNS, firewalls, proxy servers, FTP servers, and web servers.

Penetration testing can be classified into two types:**Non-destructive testing**: Identifies and verifies vulnerabilities, maps vulnerabilities with appropriate exploits, and provides proof of concept while avoiding system disruption. It does not attempt Denial-of-Service (DoS) attacks.**Destructive testing**: Includes tests such as DoS and buffer overflow attacks that may bring down a system. These tests require careful authorization and planning.

Objectives and Need for Penetration Testing

1. Identify Security Threats: It identifies threats that may expose the confidentiality of an organization.
2. Assess Organizational Security: Expert penetration testing provides a complete and detailed assessment of an organization's security.
3. Evaluate Network Efficiency: It assesses network efficiency by generating traffic and examines the security of devices such as firewalls, routers, and switches.
4. Detect New Vulnerabilities: Changes or upgrades to existing software, hardware, or network design may introduce vulnerabilities that can be detected through testing.
5. Minimize Security Risks: Penetration testing is a proactive exercise that reduces the possibility of systems being exploited by attackers.
6. Check Security Policies: It verifies whether appropriate security policies are being followed.
7. Recommend Solutions: It helps identify vulnerabilities, determine their risk levels, and suggest methods to mitigate the risks.

Example of Penetration Testing

Consider a well-known e-commerce company that earns money through online business. A hacker discovers a vulnerability in the company's website and attacks it. This may cause significant financial loss and damage the company's reputation.
By conducting penetration testing, the company can identify website vulnerabilities before malicious hackers exploit them. The tester reports the findings and recommends suitable security measures to prevent attacks.

[10*2=20]